Reporte Técnico de Actividades Práctico-Experimentales Nro. 004

Análisis comparativo de herramientas de simulación mediante un modelo común

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	[Miguel Alejandro Armas ordoñez]
Asignatura	Simulación
Ciclo	5
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje	Aplica análisis estadísticos de los datos de la simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	004
Título de la Práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del Docente	José O. Guamán Q.
Fecha	22 y 23 de octubre de 2026
Horario	Jueves 10h30–11h30; viernes 07h30–09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación. Específicamente: implementar un modelo sencillo en diferentes herramientas, comparar su sintaxis, facilidad de implementación y visualización, e identificar el ámbito de aplicación de cada herramienta.

3. Materiales, Reactivos, Equipos y Herramientas

Se utilizó un computador con sistema operativo Linux, Python 3.14, SimPy 4.x, Matplotlib 3.x y NetLogo 7.0.4 con la extensión `csv`. La ejecución final y la comparación documentada se realizaron entre Python/SimPy y NetLogo. Se empleó un editor de código, una terminal, un entorno virtual de Python y archivos CSV como mecanismo de intercambio. NumPy no fue necesario para la lógica probabilística implementada, aunque figura entre las herramientas indicadas en la guía.

4. Procedimiento / Metodología Ejecutada

Modelo y parámetros

Se adoptó el modelo epidemiológico SIR, con individuos susceptibles (S), infectados (I) y recuperados (R), conservando la población total:

$$N = S + I + R = 100. \tag{1}$$

Los parámetros de la guía fueron $S(0) = 95$, $I(0) = 5$, $R(0) = 0$, $\beta = 0{,}30$, $\gamma = 0{,}10$ y horizonte de 100 pasos. El valor de 95 susceptibles se obtiene de $100 - 5$. Como decisión adicional para favorecer la reproducción de las pruebas, se configuró una semilla de 42 en Python y NetLogo, sin asumir por ello secuencias aleatorias idénticas entre plataformas.

Cuadro 1: Parámetros empleados en Python/SimPy y NetLogo

Parámetro	Valor
Población total (N)	100
Susceptibles iniciales (S_0)	95
Infectados iniciales (I_0)	5
Recuperados iniciales (R_0^{estado})	0
Tasa de infección (β)	0.30
Tasa de recuperación (γ)	0.10
Tiempo de simulación	100 pasos
Semilla experimental	42

Fundamento matemático

Las ecuaciones diferenciales indicadas en la guía se implementaron de forma explícita en el dominio de Python y en NetLogo:

$$\frac{dS}{dt} = -\beta \frac{SI}{N}, \tag{2}$$

$$\frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I, \tag{3}$$

$$\frac{dR}{dt} = \gamma I. \tag{4}$$

El término $\beta SI/N$ expresa la tasa de nuevos contagios y γI la tasa de recuperación. En el instante inicial se obtuvieron $dS/dt = -1{,}425$, $dI/dt = 0{,}925$ y $dR/dt = 0{,}500$. El número reproductivo básico fue:

$$\mathcal{R}_0 = \frac{\beta}{\gamma} = \frac{0{,}30}{0{,}10} = 3. \tag{5}$$

Estas derivadas representan cambios esperados y pueden tener valores decimales; no deben interpretarse como el conteo exacto de personas transferidas entre estados en una corrida aleatoria. De manera complementaria, se implementó una referencia determinista mediante el método de Euler explícito con $\Delta t = 1$, que actualiza S, I y R sumando las derivadas en cada paso. Esa referencia produce valores decimales y no sustituye la simulación estocástica por individuos.

Algoritmo discreto común

Para cumplir la indicación de simular individuos en pasos discretos, se mantuvieron estados enteros y transiciones probabilísticas. Dado que la guía no especifica la cantidad de contactos por persona ni una red espacial, el proyecto emplea el supuesto uniforme:

$$P(\text{infección por paso}) = 1 - \left(1 - \frac{\beta}{N}\right)^{I}, \qquad P(\text{recuperación}) = \gamma. \tag{6}$$

Esta regla es una aproximación probabilística compatible con la incidencia esperada $\beta SI/N$ para probabilidades pequeñas; no equivale a la selección explícita de contactos de la guía. En Python se copió el estado anterior a `estados_anteriores` y se escribió el nuevo estado en `estados_siguientes`. En NetLogo se capturaron los agentsets `infectados-actuales` y `susceptibles-actuales` antes de cambiar los estados. De este modo, los cambios se aplicaron simultáneamente y los recién infectados no transmitieron durante el mismo paso en que cambiaron de estado.

Organización modular del código

Se mantuvo una arquitectura por responsabilidades, con identificadores en español: **dominio** (modelo, reglas y ecuaciones); **aplicación** (simulador y solución diferencial de referencia); **infraestructura** (exportación CSV y gráficas); y **presentación** (programa ejecutable). En NetLogo se separaron procedimientos de inicialización, ejecución, registro, importación y trazado. El modelo `SIR.nlogox` incluye la interfaz completa: cinco botones y cinco gráficos.

Implementaciones e integración

En Python se utilizó SimPy para programar la progresión del tiempo y se registró un historial con las columnas `tiempo`, `susceptibles`, `infectados` y `recuperados`. La ejecución generó `resultados_python.csv` con conteos enteros y `resultados_euler_python.csv` con la referencia determinista decimal.

En NetLogo se implementó el mismo escenario con 100 tortugas, de las cuales 5 iniciaron como infectadas. El archivo *SIR.nlogox*, ubicado en el directorio *netlogo*, contiene el código y la interfaz gráfica. Se incorporaron los botones preparar, avanzar, cargar-python, exportar-netlogo y actualizar-graficas-importadas, además de los gráficos SIR NetLogo, SIR Python, comparación de infectados, comparación de susceptibles y comparación de recuperados.

La comparación de datos importados se realiza únicamente con Python, debido a que no se incorporó un botón para cargar los resultados de Java ni un gráfico exclusivo para esta implementación.

El CSV de Python se genera por defecto en el directorio `netlogo/`, de modo que NetLogo lo lee desde la misma ubicación del modelo. El CSV de Euler es una referencia matemática y no se importa en NetLogo. NetLogo exporta su propia serie mediante el botón `exportar-netlogo`, que escribe `resultados_netlogo.csv`.

5. Resultados

Comprobación matemática y de estructura

Los tres archivos CSV contienen 101 registros correspondientes a $t = 0, 1, \ldots, 100$. El registro inicial es $(S, I, R) = (95, 5, 0)$ en los archivos de conteos y su equivalente decimal en el archivo Euler. La suma $S + I + R = 100$ se conservó en Python y NetLogo de forma exacta; en Euler se cumple dentro del error de punto flotante (100 ± 10^{-8}). Ningún individuo recuperado volvió al estado susceptible.

Cuadro 2: Resultados de las ejecuciones (semilla 42 en Python y NetLogo; Euler determinista)

Modelo	Filas	$I_{\text{máx}}$ (paso)	S_{100}	I_{100}	R_{100}
Python/SimPy	101	19 (33)	16	0	84
Euler (referencia)	101	33.0161 (19)	4.9799	0.0505	94.9695
NetLogo	101	39 (15)	6	0	94

Los valores de la referencia Euler se presentan redondeados a cuatro decimales. En Python, el máximo de infectados fue 19 en el paso 33; los infectados llegaron a cero en el paso 89 y quedaron 16 susceptibles y 84 recuperados. En NetLogo, el máximo fue 39 en el paso 15; los infectados llegaron a cero en el paso 96 y quedaron 6 susceptibles y 94 recuperados. La referencia Euler alcanzó un máximo de 33.0161 infectados en el

paso 19 y terminó con 4.9799 susceptibles, 0.0505 infectados y 94.9695 recuperados. Las diferencias entre Python y NetLogo son esperables porque cada corrida es una realización estocástica distinta y los generadores pseudoaleatorios no producen la misma secuencia, aunque se use la misma semilla. La referencia Euler describe la trayectoria media determinista y no debe coincidir exactamente con una sola corrida. La Figura 4 resume la trayectoria principal de Python/SimPy.

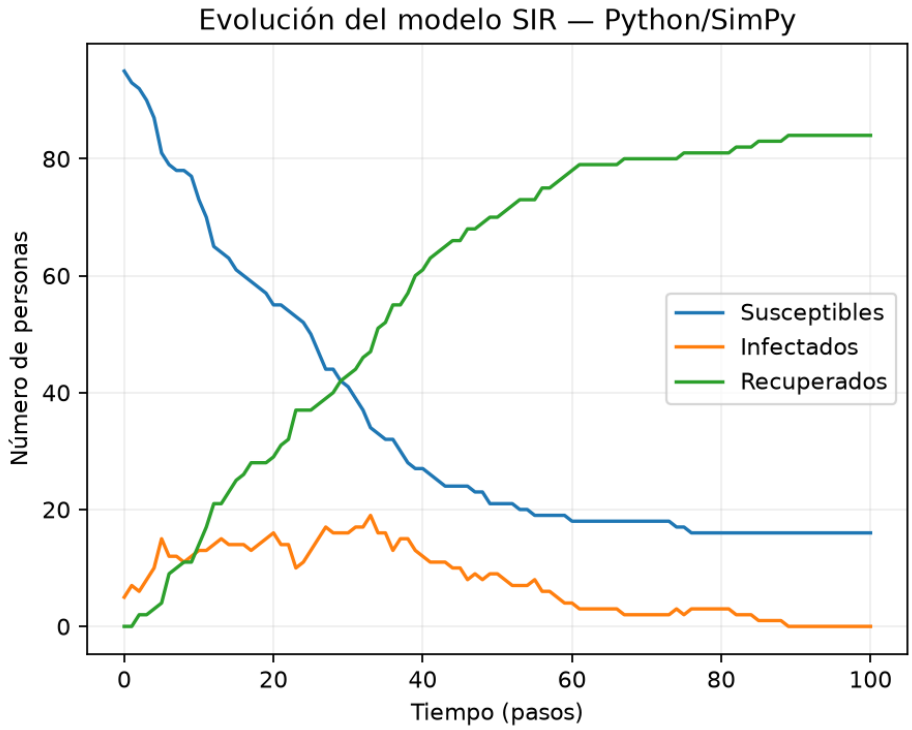


Figura 1: Evolución SIR de la simulación estocástica Python/SimPy con semilla 42.

Archivos de salida y diferencia entre CSV

Cuadro 3: Archivos CSV generados y su función

Archivo	Origen	Contenido y uso
`resultados_python.csv`	SimPy	Conteos enteros de individuos. Es el resultado principal y se importa en NetLogo.
`resultados_euler_python.csv`	Euler	Valores decimales esperados. Es una referencia matemática opcional y no se importa en NetLogo.
`resultados_netlogo.csv`	NetLogo	Conteos enteros de la simulación local. Se usa como evidencia y para revisión externa.

Los dos archivos CSV de Python comparten las columnas *tiempo*, *susceptibles*, *infectados* y *recuperados*, pero tienen distintos significados.

El archivo principal se genera mediante el módulo `infraestructura/exportador_csv.py` y conserva conteos enteros porque representa individuos de la población.

Por otra parte, el archivo correspondiente al método de Euler se genera en `presentaci`

on/consola.py, utilizando ocho decimales para aproximar la solución de las ecuaciones diferenciales del modelo SIR.

Por esta razón, los resultados de Euler pueden contener valores como $0{,}0505$ infectados, lo cual representa una cantidad estimada por el modelo matemático y no una persona individual.

Finalmente, el archivo de Euler no reemplaza al CSV principal ni se utiliza para realizar las comparaciones en NetLogo.

Comparación de herramientas

Cuadro 4: Comparación técnica de las implementaciones ejecutadas

Característica	**NetLogo**	**Python/SimPy**
Facilidad de implementación	Procedimientos sencillos y controles visuales	Lectura clara con módulos y funciones
Manejo de agentes	Tortugas con estado individual	Individuos representados en una lista de estados
Modelo de tiempo	`ticks` discretos	Entorno SimPy con `timeout(1)`
Generación de datos	Listas internas y exportación CSV	Historial y exportador CSV
Visualización	Gráficos y agentes integrados	Matplotlib
Flexibilidad	Alta para modelos basados en agentes	Alta para experimentos y análisis
Integración	Lectura del CSV de Python	Exportación de CSV
Tiempo de ejecución	No medido	No medido

La comparación se realizó entre Python/SimPy y NetLogo, que son las implementaciones ejecutadas. No se reportan tiempos de ejecución porque no se registraron mediciones comparables en el mismo equipo. El modelo NetLogo se ejecutó durante 100 pasos y exportó su CSV; la interfaz contiene los cinco gráficos solicitados y no incluye comparación con Java.

6. Preguntas de Control

1. **¿Qué representa el modelo SIR?** Representa la evolución de una población durante la propagación de una infección, separando a los individuos en susceptibles, infectados y recuperados.

2. **¿Qué representan los estados S, I y R?** S son las personas que pueden infectarse; I, las que cestan infectadas y pueden transmitirla; R, las que han dejado el estado infeccioso y no vuelven a ser susceptibles dentro del modelo clásico utilizado.

3. **¿Qué función cumplen los parámetros β y γ?** β controla la transmisión y γ la recuperación. En este proyecto sus valores son 0.30 y 0.10, respectivamente.

4. **¿Por qué es importante utilizar los mismos parámetros en las implementaciones?** Porque así las dos herramientas simulan las mismas condiciones, como la cantidad de personas, la tasa de infección y el tiempo. De esta manera, podemos comparar los resultados y saber si las diferencias se deben a cómo funciona cada herramienta y no a que se utilizaron valores distintos.

5. **¿Por qué Python y NetLogo pueden producir resultados diferentes utilizando el mismo modelo?** Porque generan números pseudoaleatorios con algoritmos y consumos de secuencia diferentes; además, cada corrida es una sola trayectoria estocástica. La semilla 42 garantiza reproducibilidad dentro de cada plataforma, no una sincronización entre plataformas.

6. **¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?** Permite definir agentes con estados y comportamientos propios, visualizar sus cambios y controlar el experimento mediante botones, monitores y gráficos.

7. **¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?** Permite expresar procesos que avanzan en el tiempo, esperar eventos y coordinar actividades discretas sin gestionar manualmente un reloj de simulación de bajo nivel.

8. **¿Qué función cumple el archivo CSV en la integración Python → NetLogo?** Sirve como formato común para trasladar la serie de tiempo sin acoplar directamente los programas; NetLogo lee las columnas y las representa en los gráficos de comparación.

9. **¿Qué ventajas presenta separar la generación de datos de la visualización?** Facilita reutilizar un motor de simulación con distintas interfaces, probar los cálculos por separado, intercambiar resultados y localizar errores.

7. Conclusiones

La implementación modular permitió representar un escenario SIR común con una población inicial de 100 personas y parámetros equivalentes en Python/SimPy y NetLogo. La separación entre dominio, aplicación, infraestructura y presentación facilitó identificar las reglas matemáticas, las transiciones de individuos y los archivos de salida.

Las ecuaciones diferenciales describieron las tasas esperadas de contagio y recuperación, mientras que los simuladores trabajaron con cambios probabilísticos de individuos completos. La referencia Euler permitió disponer de una trayectoria determinista de comparación, pero no sustituyó el resultado estocástico principal. Esta distinción fue necesaria para interpretar correctamente las gráficas y justificar las discrepancias entre trayectorias.

Los resultados conservaron la población total y mostraron el mismo patrón cualitativo: aumento inicial de infectados, pico y posterior recuperación. Python alcanzó un máximo de 19 infectados en el paso 33 y terminó con 16 susceptibles y 84 recuperados; NetLogo alcanzó un máximo de 39 en el paso 15 y terminó con 6 susceptibles y 94 recuperados; la referencia Euler alcanzó 33.0161 infectados en el paso 19. Las diferencias en el momento y la magnitud del pico confirman la influencia de la aleatoriedad y la necesidad de realizar varias réplicas para formular conclusiones cuantitativas más sólidas.

NetLogo resultó especialmente útil para observar agentes y comparar curvas; Python favoreció el análisis y la organización de experimentos. La comparación se considera completa para las dos implementaciones ejecutadas y permite reconocer sus ventajas y limitaciones, aunque no se midieron tiempos de ejecución bajo condiciones equivalentes.

8. Recomendaciones

Se recomienda ejecutar varias réplicas con diferentes semillas y resumir el máximo de infectados, su momento y el número final de recuperados mediante medias y medidas de dispersión. Conviene documentar una regla única de contactos para ambas implementaciones y mantener exactamente la misma secuencia de actualización de estados.

También es aconsejable conservar la interfaz de `SIR.nlogox` con los cinco gráficos solicitados, incorporar capturas de la ejecución local, registrar el tiempo de ejecución de cada plataforma en el mismo equipo y guardar los CSV originales junto con los parámetros de cada experimento. Se debe ejecutar Python como paquete mediante `python -m sir.presentacion.consola` desde el directorio `python_sir`, evitando el error de importación del módulo `sir`.

9. Bibliografía / Referencias

[1] NetLogo, *NetLogo User Manual*. [En línea]. Disponible: `https://docs.netlogo.org/`

[2] SimPy, *SimPy Documentation*. [En línea]. Disponible: `https://simpy.readthedocs.io/`

[3] Python Software Foundation, *Python Documentation*. [En línea]. Disponible: `https://docs.python.org/3/`

10. Declaración de uso de Inteligencia Artificial

Durante el desarrollo de esta práctica se utilizó ChatGPT como herramienta de apoyo para la implementación y revisión del modelo de simulación SIR. Su uso estuvo enfocado principalmente en la generación de gráficas para representar los resultados y en la orientación para desarrollar el código en NetLogo, debido al desconocimiento inicial de su sintaxis, estructura y funcionamiento.

Asimismo, se solicitó apoyo para verificar el código implementado, identificar posibles errores y reconocer aspectos que podían mejorarse, especialmente en la organización modular, la aplicación de las fórmulas matemáticas y la exportación de resultados en archivos CSV.

La inteligencia artificial se utilizó como una herramienta complementaria de consulta y asistencia técnica. Su participación no reemplazó el análisis, la interpretación de resultados ni la toma de decisiones durante el desarrollo de la práctica.

11. Prompts Usados

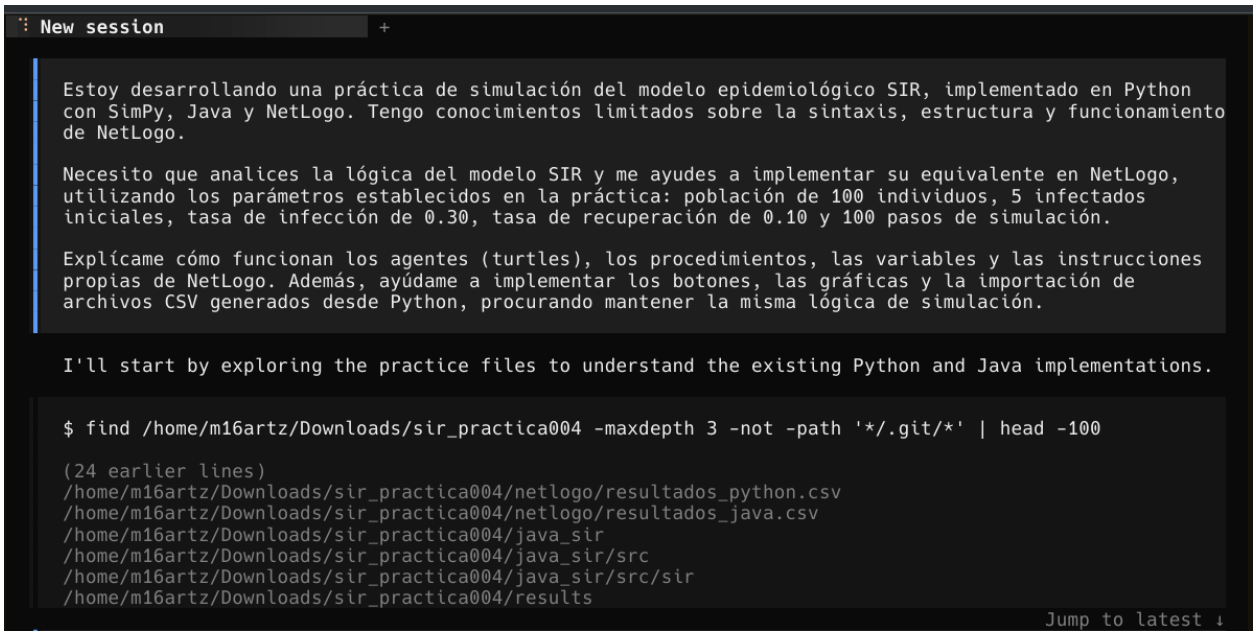


Figura 2: Prompt para la implementacion con NetLogo.

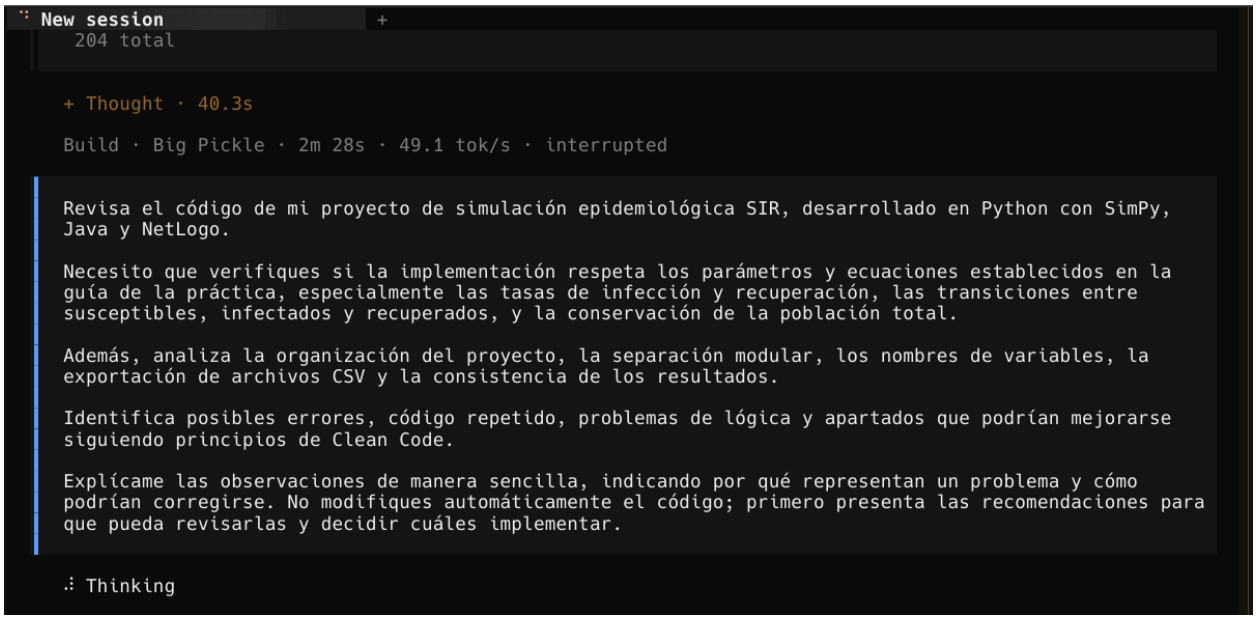


Figura 3: Prompt para la revision del codigo.

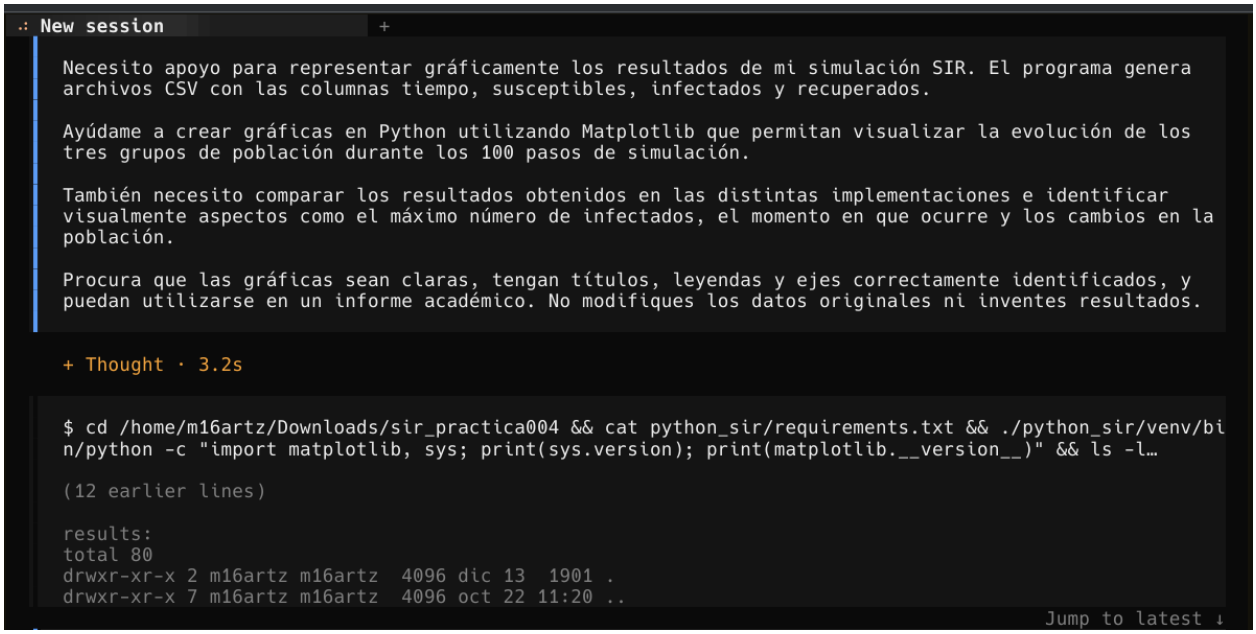


Figura 4: Prompt para la realizacion de la grafica.